



5

Lab

DLL Injection

Thực hành Cơ chế hoạt động của Mã độc

Lưu hành nội bộ 2026

A. TỔNG QUAN

DLL Injection đã trở thành một kỹ thuật khá phổ biến trong việc chỉnh sửa phần mềm. Phương thức này giúp ta điều chỉnh các tính năng của một ứng dụng mà không làm ảnh hưởng trực tiếp đến mã thực thi (binary).

Bài thực hành này đặt ra các ngữ cảnh thực hiện kỹ thuật DLL Injection, từ ví dụ đơn giản để nắm cơ chế hoạt động đến ngữ cảnh thông dụng là "hack" một trò chơi để chỉnh sửa tài nguyên và thậm chí logic của trò chơi.

B. CHUẨN BỊ MÔI TRƯỜNG

- Mã nguồn game Age of Empires và một số công cụ thực hiện DLL Injection (GVTH cung cấp).
- Máy hệ điều hành Windows, cài đặt công cụ Cheat Engine ().
- Visual Studio.

C. THỰC HÀNH

C.1 Kịch bản 1: Kỹ thuật DLL Injection cơ bản

DLL Injection là kỹ thuật cho phép kẻ tấn công chèn thêm code vào các tiến trình đang chạy để hướng tới thực thi các đoạn mã tùy ý hoặc qua mặt các biện pháp bảo vệ, leo thang đặc quyền. Kỹ thuật này thực hiện chèn thêm mã vào không gian địa chỉ của một tiến trình đang thực thi. Khi đó, mã nguồn chạy trong ngữ cảnh của tiến trình khác có thể cho phép truy xuất vùng nhớ của tiến trình, các tài nguyên hệ thống hoặc các quyền hạn cụ thể.

Để thực hiện kỹ thuật này cần chuẩn bị:

- **Tập tin DLL** chứa mã nguồn cần thực thi, trong ngữ cảnh mã độc thường chứa các payload tấn công.
- **Injector** có nhiệm vụ xác định tiến trình mục tiêu để chèn DLL đã lập trình và thực thi nó trong không gian vùng nhớ của tiến trình này.

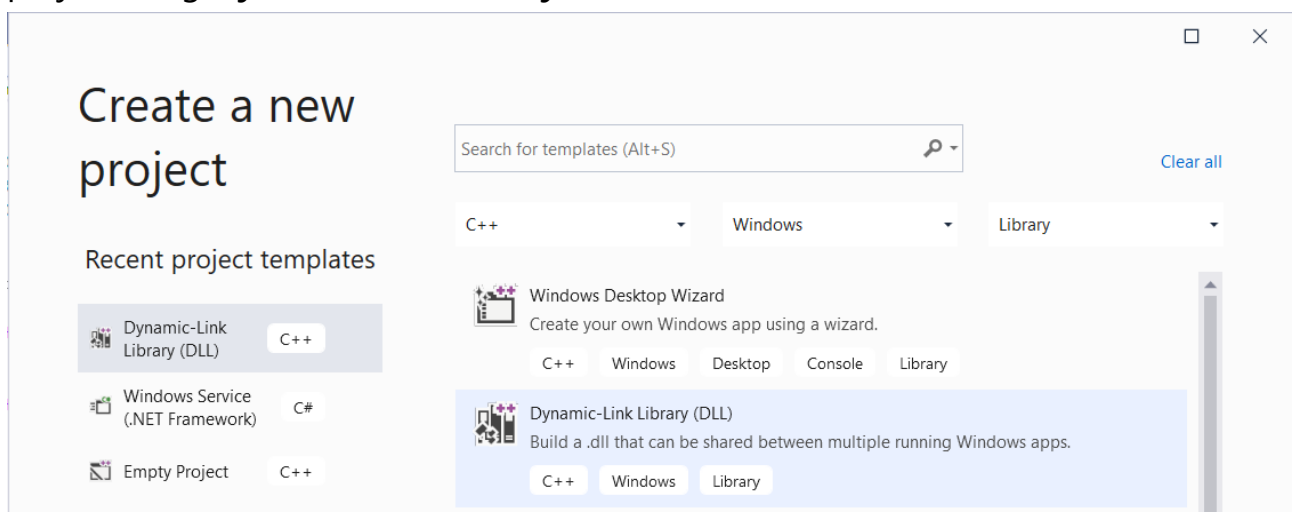
C.1.1 Hướng dẫn DLL Injection vào tiến trình

Yêu cầu 1.1: Sinh viên thực hiện theo hướng dẫn để thực hiện DLL Injection vào tiến trình Notepad.

C.1.1.1 Chuẩn bị tập tin DLL

a) Tạo project DLL

Vào Visual Studio, chọn **File > New > Project**. Trong cửa sổ **Create a new project**, chọn các điều kiện lọc **C++**, **Windows** và **Library** như bên dưới, sau đó chọn project dạng **Dynamic-Link Library (DLL)** để tạo các thư viện DLL.



Đặt tên cho project, tên này mặc định cũng là tên của tệp DLL sau khi build.

Configure your new project

Dynamic-Link Library (DLL) C++ Windows Library

Project name
DemoDLL

Location
G:\My Drive\WORKING\2. Giảng dạy\GIANG DẠY\NT230\CO CHE HD MA DOC\LABS\LAB NEW\L

Solution
Create new solution

Solution name ⓘ
DemoDLL

☐ Place solution and project in the same directory

Back Create

Project được tạo sẽ mặc định có một tệp tin `dllmain.cpp` chứa mã nguồn thực thi của tệp tin DLL trong hàm `DllMain`, trong đó có thể thấy một nhóm các trường hợp `switch-case` tương ứng với một số sự kiện liên quan đến tệp tin DLL.

```

1 // dllmain.cpp : Defines the entry point for the DLL application.
2 #include "pch.h"
3
4 BOOL APIENTRY DllMain( HMODULE hModule,
5                       DWORD ul_reason_for_call,
6                       LPVOID lpReserved
7                       )
8 {
9     switch (ul_reason_for_call)
10    {
11        case DLL_PROCESS_ATTACH:
12        case DLL_THREAD_ATTACH:
13        case DLL_THREAD_DETACH:
14        case DLL_PROCESS_DETACH:
15            break;
16    }
17    return TRUE;
18 }
19
  
```

Một số sự kiện thường thấy ứng với các case:

- **DLL_PROCESS_ATTACH**: DLL đang được load vào vùng nhớ ảo của tiến trình khi tiến trình khởi chạy hoặc dùng hàm `LoadLibrary` để load thư viện động. Có thể tận dụng sự kiện này để khởi tạo các tài nguyên liên quan và thực hiện các tác vụ khi DLL vừa được load vào tiến trình.
- **DLL_PROCESS_DETACH**: DLL đang được unload khỏi vùng nhớ ảo của tiến trình gọi, do load không thành công hoặc tiến trình kết thúc hoặc gọi hàm `FreeLibrary()`.
- **DLL_THREAD_ATTACH/DLL_THREAD_DETACH**: khi tiến trình tạo một luồng (thread) mới hoặc kết thúc một luồng (thread).

b) Thêm chức năng chính của DLL

Để tránh ảnh hưởng đến luồng thực thi chính của tiến trình sử dụng, ta có thể tạo một thread (luồng) riêng để thực thi hoạt động của DLL. Ta tạo tiến trình trong case **DLL_PROCESS_ATTACH** để luồng này được tạo ngay khi DLL được load vào tiến trình.

Ví dụ bên dưới, ngay khi được load vào tiến trình, DLL sẽ hiển thị **MessageBox Injected!** Để báo hiệu đã thêm thành công vào tiến trình, sau đó tạo một thread (luồng) để thực thi chức năng hàm **MyThreadFunction**, hoạt động chính là hiển thị Message box sau mỗi 30 giây.

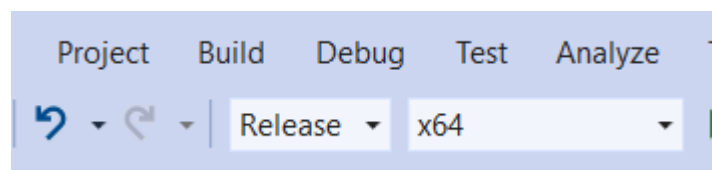
```

1 // dllmain.cpp : Defines the entry point for the DLL application.
2 #include "pch.h"
3 #include "dllmain.h"
4 #include "stdio.h"
5 #include "time.h"
6 #include "windows.h"
7 #include "string.h"
8
9 DWORD WINAPI MyThreadFunction(LPVOID lpParam) {
10     while (1) {
11         MessageBoxA(NULL, "Hello Class", "Info", NULL);
12         Sleep(30000);
13     }
14     return 0;
15 }
16
17 BOOL APIENTRY DllMain( HMODULE hModule,
18                       DWORD ul_reason_for_call,
19                       LPVOID lpReserved
20                       )
21 {
22     switch (ul_reason_for_call)
23     {
24     case DLL_PROCESS_ATTACH:
25         MessageBoxA(NULL, "Q221-Group 1 Injected!", "Info", MB_OK);
26         CreateThread(NULL, 0, MyThreadFunction, NULL, 0, NULL);
27         break;
28     case DLL_THREAD_ATTACH:
29     case DLL_THREAD_DETACH:
30     case DLL_PROCESS_DETACH:
31         break;
32     }
33     return TRUE;
34 }

```

c) Build tạo DLL

Tùy chỉnh phần cấu hình biên dịch thành chế độ Release và kiến trúc x86 hoặc x64 tương ứng tùy vào môi trường của tiến trình mục tiêu sẽ chèn DLL. Sau đó chọn **Build > Build Solution**.



Cửa sổ Output hiển thị kết quả build và lỗi (nếu có). Nếu thành công, ta sẽ có được một tập tin .DLL.

```

Output
Show output from: Build
Build started...
1>----- Build started: Project: DemoDLL, Configuration: Release x64 -----
1>pch.cpp
1>dllmain.cpp
1>Generating code
1>Previous IOBJ not found, fall back to full compilation.
1>All 2 functions were compiled because no usable IPDB/IOBJ from previous compilation was found.
1>Finished generating code
1>DemoDLL.vcxproj -> E:\NT230\Notepad\DemoDLL\x64\Release\DemoDLL.dll
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====

```

C.1.1.2 Chuẩn bị Injector

Sinh viên sử dụng mã nguồn Injector.cpp được cung cấp sẵn, trong đó thực hiện các bước tìm kiếm tiến trình và chèn DLL đã chuẩn bị vào tiến trình tương ứng.

a) Xác định tiến trình mục tiêu và tập tin DLL

Tiến trình mục tiêu là notepad.exe và dll được gán là tên của tập DLL được tạo. Hàm **getProcId** thực hiện tìm kiếm Process ID dựa trên tên tiến trình được cung cấp.

```

75     HANDLE hProcess;
76     LPVOID pszLibFileRemote = NULL;
77     HANDLE handleThread;
78     const wchar_t* process = L"notepad.exe";
79     int pID = getProcId(process);
80
81     char dll[] = "DemoDLL.dll";
82     if (!exist(dll)) {
83         std::cout << "debuginfo: Invalid DLL path!" << std::endl;
84     }
85     char dllPath[MAX_PATH] = { 0 }; // full path of DLL
86     GetFullPathNameA(dll, MAX_PATH, dllPath, NULL);
87     std::cout << "debuginfo: Full DLL path: " << dllPath << std::endl;

```

b) Chèn DLL vào tiến trình đang thực thi

? Sinh viên quan sát và giải thích hoạt động của đoạn mã bên dưới dùng để hiện thực kỹ thuật DLL Injection?

```

90 hProcess = OpenProcess(PROCESS_QUERY_INFORMATION | PROCESS_CREATE_THREAD | PROCESS_VM_OPERATION | PROCESS_VM_WRITE, 0, pID);
91
92 if (hProcess) {
93     pszLibFileRemote = VirtualAllocEx(hProcess, NULL, strlen(dllPath) + 1, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
94 }
95 else {
96     std::cout << "error: Could not open process handle with id:" << pID << std::endl;
97 }
98 if (pszLibFileRemote == NULL) std::cout << "error: Cannot allocate memory" << std::endl;
99
100 int isWriteOK = WriteProcessMemory(hProcess, pszLibFileRemote, dllPath, strlen(dllPath) + 1, NULL);
101 if (!isWriteOK) std::cout << "error: Failed to write" << std::endl;
102
103 handleThread = CreateRemoteThread(hProcess, NULL, NULL, (LPTHREAD_START_ROUTINE)LoadLibraryA, pszLibFileRemote, NULL, NULL);
104 if (handleThread == NULL) {
105     std::cout << "error: Failed to create thread" << std::endl;
106     ErrorExit(_T("CreateRemoteThread"));
107 }
108
109 WaitForSingleObject(handleThread, INFINITE);
110 CloseHandle(handleThread);
111 VirtualFreeEx(hProcess, dllPath, 0, MEM_RELEASE);
112 CloseHandle(hProcess);

```

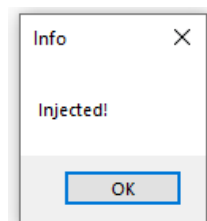
c) Build tạo Injector

Sử dụng chức năng **Build > Build Solution** với tùy chỉnh Release và kiến trúc tương tự DLL để tạo tập tin thực thi cho Injector.

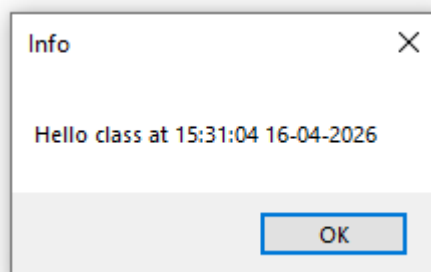
C.1.1.3 Thực hiện DLL Injection vào tiến trình mục tiêu

Trên máy victim, cần đảm bảo tiến trình mục tiêu đang chạy để thực hiện DLL injection nhằm thực thi hoạt động mong muốn.

Chạy Injector, nếu thu được thông báo như bên dưới là đã inject DLL vào tiến trình thành công.



Chức năng chính đã lập trình của DLL khi đó sẽ thực thi và hiển thị Message Box.



Yêu cầu 1.2: Sinh viên quan sát máy victim sau khi thực hiện thành công DLL Injection và trả lời các câu hỏi:

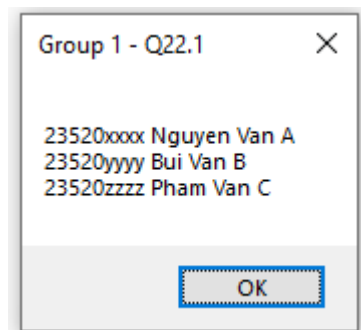
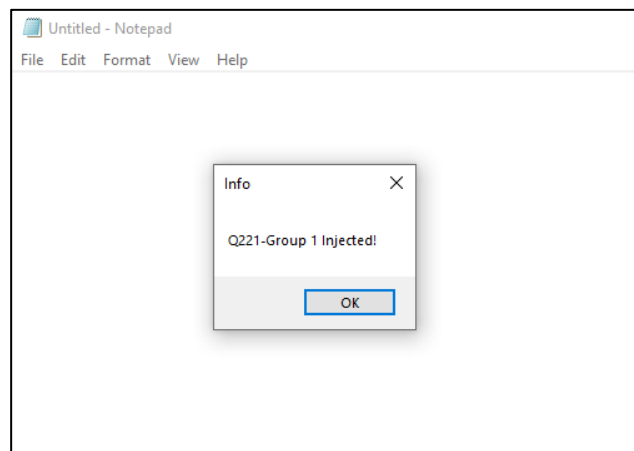
- a) Thực hiện DLL Injection với thread thực thi như trên thì khi tiến trình mục tiêu kết thúc, DLL còn hoạt động không? Giải thích?
- b) Nếu còn hoạt động, làm thế nào để dừng hoạt động và unload DLL này?

Có thể kiểm tra tồn tại của DLL và tiến trình liên quan bằng công cụ Process Explorer.

C.1.2 Yêu cầu: DLL Injection vào tiến trình Notepad

Yêu cầu 1.3: Sinh viên tìm hiểu và thực hiện tương tự để DLL injection vào tiến trình của Notepad, sao cho:

- Tên tệp tin DLL cần lưu là Q221-GroupX-notepad.dll
- Các chức năng cần thực hiện thêm:
 - + Hiện Message box "Q221-Group X Injected!" khi inject thành công vào tiến trình.
 - + Hiện Message box chứa thông tin nhóm mỗi khi nhấn F5.



C.2 Kịch bản 2: Sử dụng DLL Injection để hack trò chơi Age of Empires

Phần thực hành này lấy ngữ cảnh “hack” trò chơi Age of Empire: Rise of Rome - một tựa game đình đám ra mắt lần đầu tiên vào khoảng hơn hai thập kỷ trước. Cụ thể, mục tiêu hack là sử dụng kỹ thuật DLL Injection để can thiệp thay đổi 1 trong các tài nguyên của trò chơi là “thức ăn”.

Dưới góc độ lập trình, một đại lượng như vậy trong trò chơi sẽ có giá trị được lưu trong một biến nào đó, và khi thực thi sẽ tồn tại trên một ô nhớ nhất định. Để thay đổi hay “hack” được giá trị này thành giá trị ta mong muốn thì cần tiếp cận được vị trí bộ nhớ lưu trữ biến đó và ghi đè giá trị.

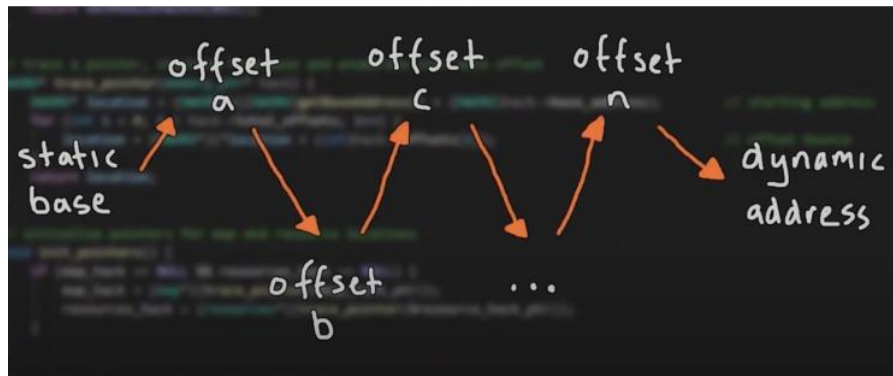
Yêu cầu 2: Sinh viên tìm hiểu và thực hiện các tùy chỉnh cần thiết trong mã nguồn được cung cấp để hiện thực hack game AOE. Ràng buộc:

- Tên tệp tin DLL cần lưu là Q221-GroupX-AOE.dll
- Tăng giá trị biến thức ăn lên 500 mỗi lần nhấn phím F9.

C.2.1 Kỹ thuật nền tảng: Cơ chế static pointers

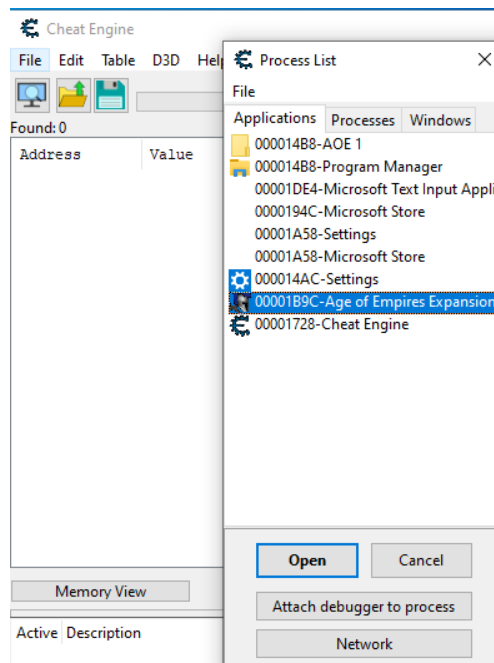
Khi hack biến “thức ăn”, khó khăn nằm ở việc xác định được vị trí bộ nhớ lưu trữ biến trong không gian bộ nhớ của chương trình. Và do cơ chế ngẫu nhiên hóa bộ nhớ của nhiều hệ điều hành, không gian bộ nhớ ảo để chạy ứng dụng không phải lúc nào cũng cố định và giống nhau giữa những lần chạy.

Để giải quyết vấn đề này, ta có thể tìm trong chương trình các vị trí static pointer hay static base, thường cố định hoặc dễ xác định mỗi lần chương trình chạy, đóng vai trò là một điểm khởi đầu tin cậy để từ đó truy cập đến các dữ liệu mà có thể sẽ được cấp phát linh động hơn. Bằng cách đọc các địa chỉ ở các static pointer này và áp dụng một số offset định nghĩa trước, ta có thể tiến tới vị trí dữ liệu động mục tiêu. Việc tìm địa chỉ như vậy được gọi là tạo pointer map.

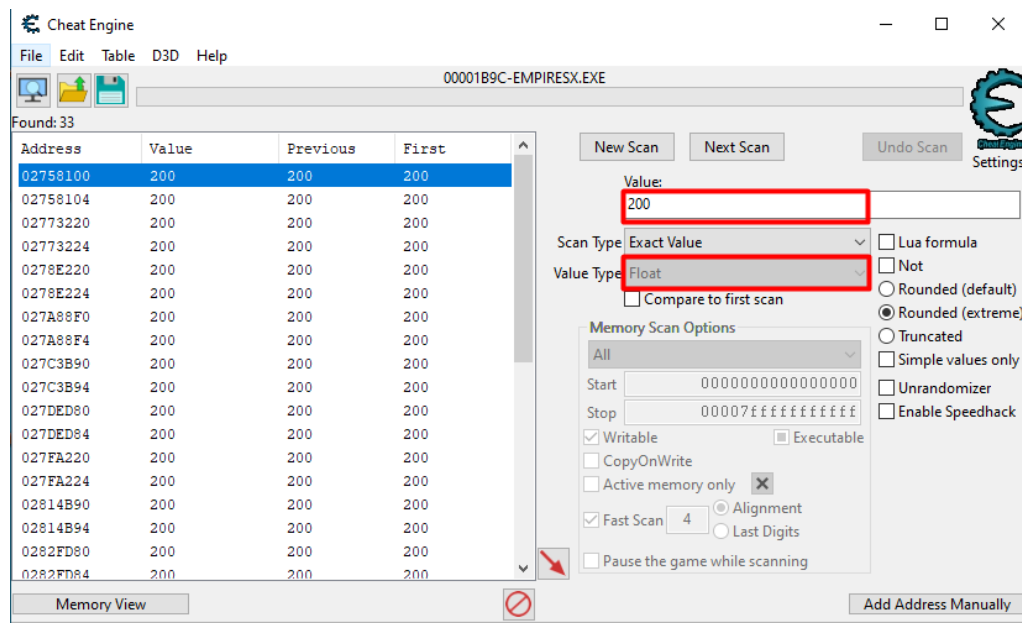


Ở đây ta sẽ sử dụng một công cụ là Cheat Engine có chức năng quét bộ nhớ của chương trình để xác định vị trí của đại lượng “thức ăn” trong game dựa trên cơ chế này.

Mở tiến trình của game với công cụ Cheat Engine.



Nhập giá trị hiện tại của biến thức ăn để làm cơ sở cho công cụ quét trong bộ nhớ và cung cấp danh sách các địa chỉ khả nghi có liên quan.



Do có nhiều kết quả, thực hiện thêm 1 số thao tác để xác định vị trí của biến thức ăn:

- **Cách 1:** Vào game thực hiện các thao tác để giá trị thức ăn thay đổi, sau đó quan sát các giá trị được tô đỏ (vừa thay đổi) trên cửa sổ Cheat Engine hoặc sử dụng giá trị mới cho lần scan kế tiếp.

- **Cách 2:** Nhấp đúp chuột vào dòng bất kỳ, ở phần hiển thị bên dưới có thể thay đổi giá trị tùy ý. Sau đó quan sát lại giá trị trong game để biết đâu là vị trí lưu chính xác.

Khi đã xác định được vị trí biến thức ăn, thực hiện tạo pointer map đến địa chỉ này bằng cách chọn option **Pointer scan for this address**. Có thể thu được kết quả scan như bên dưới, ở đây ta quan tâm đến các kết quả sử dụng 4 offset.

Pointer scan : dhd.PTR								
File Distributed pointer scan Pointer scanner								
Float	Pointer paths:832573							
Base Address	Offset 0	Offset 1	Offset 2	Offset 3	Offset 4	Offset 5	Offset 6	Points to:
"EMPIRESX.EXE"+003D70...	56C	650						028F75B0 = 1150
"EMPIRESX.EXE"+003D70...	568	18	6B0					028F75B0 = 1150
"EMPIRESX.EXE"+003D70...	568	50	660					028F75B0 = 1150
"EMPIRESX.EXE"+002A68...	88	56C	650					028F75B0 = 1150
"EMPIRESX.EXE"+003C49...	88	56C	650					028F75B0 = 1150
"EMPIRESX.EXE"+003D70...	56C	24	5C0					028F75B0 = 1150
"EMPIRESX.EXE"+003D70...	82C	880	178	0				028F75B0 = 1150
"EMPIRESX.EXE"+003D70...	82C	884	E8	0				028F75B0 = 1150
"EMPIRESX.EXE"+00188144	540	10C	50	0				028F75B0 = 1150

Sau khi có được kết quả quét bộ nhớ, ta có thể nhấp đúp chuột vào 1 dòng bất kỳ để quan sát pointer map để dẫn đến địa chỉ đang lưu đại lượng mục tiêu.

Active	Description	Address	Type	Value
☐	No description	028F75B0	Float	1150
☐	pointerscan result	P->028F75B0	Float	1150
☐	pointerscan result	P->028F75B0	Float	1150
☐	pointerscan result	P->028F75B0	Float	1150

Nhấp chuột phải và chọn **Change record > Address** để xem các giá trị chi tiết của pointer map. Dựa vào các kết quả ở hình dưới, Cheat Engine đã tìm được 1 pointer map – một “công thức” dựa trên nhiều offset để từ một static base đến vị trí đại lượng mục tiêu.

Từ kết quả trên ta được công thức xác định vị trí của biến thức ăn (lưu ý các vị trí truy xuất giá trị ô nhớ với []). Trong đó: base = 0x00188144, offset1 = 0x100, offset2 = 0x40, offset3 = 0x40, offset4 = 0x860.

$$*(*(*(\text{Empiresx.exe} + \text{base} + \text{offset1}) + \text{offset2}) + \text{offset3}) + \text{offset4}$$

Lưu ý đây chỉ là một trong nhiều “công thức” hợp lệ. Có nhiều cách để xác định địa chỉ của đại lượng.

Vậy chỉ cần xác định được Địa chỉ load của module Empiresx.exe và cộng theo công thức ta đã có thể đến và chỉnh được đại lượng thức ăn!

C.2.2 Hiện thực DLL Injection để hack giá trị thức ăn của AOE

Quan sát và tham khảo các tập tin trong GameHacking.zip được cung cấp.

C.2.2.1 Tạo tập tin DLL có chức năng tăng biến thức ăn

- Mã nguồn thực hiện tìm kiếm và thay đổi biến thức ăn

Tham khảo tập tin Source.cpp trong thư mục GameHacking, hàm main trong tập tin này là mã nguồn hiện thực việc tính toán địa chỉ của đại lượng thức ăn dựa trên "công thức" hay pointer map ở trên.

- Tìm tiến trình đang chạy của game AOE

```
int main() {

    HWND hGameWindow = FindWindow(NULL, L"Age of Empires Expansion");
    if (hGameWindow == NULL) {
        std::cout << "Start the game!" << std::endl;
        return 0;
    }
    DWORD pID = NULL; // ID of our Game
    GetWindowThreadProcessId(hGameWindow, &pID);
    HANDLE processHandle = NULL;
    processHandle = OpenProcess(PROCESS_ALL_ACCESS, FALSE, pID);
    if (processHandle == INVALID_HANDLE_VALUE || processHandle == NULL) { // error handling
        std::cout << "Failed to open process" << std::endl;
        return 0;
    }
}
```

- Tính toán địa chỉ của biến thức ăn: phần này sử dụng công thức pointer map đã tìm được ở phần C.2.1.

```
TCHAR gameName[13];
wcscpy_s(gameName, 13, L"Empires.exe");

DWORD gameBaseAddress = GetModuleBaseAddress(gameName, pID);

std::cout << "debuginfo: gameBaseAddress = " << gameBaseAddress << std::endl;

DWORD offsetGameToBaseAddress = 0x003C4B18;
std::vector<DWORD> pointsOffsets{ 0x3c, 0x100, 0x50, 0x0 };

DWORD baseAddress = NULL;

//Get value at gamebase+offset -> store it in baseAddress
ReadProcessMemory(processHandle, (LPVOID)(gameBaseAddress+ offsetGameToBaseAddress), &baseAddress, sizeof(baseAddress), NULL);
std::cout << "debuginfo: baseaddress = " << std::hex << baseAddress << std::endl;

DWORD pointsAddress = baseAddress; //the Address we need -> change now while going through offsets
for (int i = 0; i < pointsOffsets.size() - 1; i++) // -1 because we dont want the value at the last offset
{
    ReadProcessMemory(processHandle, (LPVOID)(pointsAddress + pointsOffsets.at(i)), &pointsAddress, sizeof(pointsAddress), NULL);
    std::cout << "debuginfo: Value at offset = " << std::hex << pointsAddress << std::endl;
}
pointsAddress += pointsOffsets.at(pointsOffsets.size() - 1); //Add Last offset -> done!!
```

- Đọc và in giá trị hiện tại và nhận giá trị mới để ghi đè

```
float currentPoint = 0;

std::cout << sizeof(currentPoint) << std::endl;
ReadProcessMemory(processHandle, (LPVOID)(pointsAddress), &currentPoint, sizeof(currentPoint), NULL);
std::cout << "The last address is:" << pointsAddress << std::endl;
std::cout << "Current value is:" << currentPoint << std::endl;
// "UI"
std::cout << "Age of Empires Hack" << std::endl;

std::cout << "How many points you want?" << std::endl;
float newPoints = 0;
std::cin >> newPoints;
WriteProcessMemory(processHandle, (LPVOID)(pointsAddress), &newPoints, 4, 0);
```

Trong đoạn mã này có in ra giá trị hiện tại của biến thức ăn và nhận giá trị mới từ người dùng để gán cho biến này.

- Mã nguồn của tập tin DLL

Tập tin dllmain.cpp là mã nguồn tham khảo của tập tin DLL. Trong đó MainThread sẽ định nghĩa chức năng chính của DLL, mặc định hiện tại là một vòng lặp vô tận lắng nghe sự kiện nhấn phím F6 để hiện 1 cửa sổ thông báo.

```
1 // dllmain.cpp : Defines the entry point for the DLL application.
2 #include "pch.h"
3 #include <Windows.h>
4
5 DWORD WINAPI MainThread(LPVOID param) {
6     while (true) {
7         if (GetAsyncKeyState(VK_F6) & 0x80000) {
8             MessageBoxA(NULL, "F6 Pressed!", "F6 Pressed!", MB_OK);
9         }
10        Sleep(100);
11    }
12    return 0;
13 }
14
15
16 BOOL APIENTRY DllMain( HMODULE hModule,
17                       DWORD ul_reason_for_call,
18                       LPVOID lpReserved
19                       )
20 {
21     switch (ul_reason_for_call)
22     {
23     case DLL_PROCESS_ATTACH:
24         MessageBoxA(NULL, "DLL Injected!", "DLL Injected!", MB_OK);
25         CreateThread(0, 0, MainThread, hModule, 0, 0);
26     case DLL_THREAD_ATTACH:
27     case DLL_THREAD_DETACH:
28     case DLL_PROCESS_DETACH:
29         break;
30     }
31     return TRUE;
32 }
```

? Cần điều chỉnh và kết hợp đoạn mã của 2 file trên như thế nào để theo dõi phím F9 và thực hiện tự động cập nhật biến thức ăn của tiến trình AOE lên 500?

C.2.2.2 Tạo Injector để thực hiện DLL Injection

Mã nguồn Injector.cpp có chức năng chính ở hàm main thêm DLL vào tiến trình đang chạy của game AOE.

Một số giá trị mặc định cần điều chỉnh:

- Tên tiến trình AOE (chú ý hoa thường).
- Tên tập tin DLL.

```

72 int main()
73 {
74     HANDLE hProcess;
75     LPVOID pszLibFileRemote = NULL;
76     HANDLE handleThread;
77     const wchar_t* process = L"Empiresx.exe";
78     int pID = getProcId(process);
79
80     char dll[] = "AOEResourceHack.dll";
81     if (!exist(dll)) {
82         std::cout << "debuginfo: Invalid DLL path!" << std::endl;
83     }
84     char dllPath[MAX_PATH] = { 0 }; // full path of DLL
85     GetFullPathNameA(dll, MAX_PATH, dllPath, NULL);
86     std::cout << "debuginfo: Full DLL path: " << dllPath << std::endl;
87

```

D. YÊU CẦU

- Sinh viên tìm hiểu và thực hành theo hướng dẫn theo nhóm **tối đa 3 sinh viên**.
- Báo cáo kết quả chi tiết những việc (**Report**) đã thực hiện, quan sát thấy và kèm ảnh chụp màn hình kết quả (nếu có); giải thích (nếu có), video demo (điểm cộng).

Báo cáo:

- File **.PDF**, **ghi rõ** nhóm sinh viên thực hiện.
- Đặt tên theo định dạng: [Mã lớp]-Lab4_MSSV1-MSSV2-MSSV3.pdf
Ví dụ: [NT230.Q2x.ATTT.y]-Lab4_2352xxxx-2352yyyy-2352zzzz.pdf
- Nếu báo cáo có nhiều file, nén tất cả file vào file .ZIP với cùng tên file báo cáo.
- Nộp file báo cáo trên theo thời gian đã thống nhất tại courses.uit.edu.vn.

Đánh giá: Sinh viên hiểu và tự thực hiện được bài thực hành. Khuyến khích:

- Có nội dung mở rộng, ứng dụng trong kịch bản phức tạp hơn, có đóng góp xây dựng bài thực hành.

Bài sao chép, trể,... sẽ được xử lý tùy mức độ vi phạm.
-HẾT-